

Analytics Engineering: Session 16

Practice Session IV: Macros, Python Models & Testing

Goal

Consolidate yesterday's session: write macros from scratch, build a Python model, and add a full test suite to your work.

By the end of this session you will have:

1. A new macro using Jinja `if/elif/else` and default arguments
2. A Python model (Pandas) that analyzes website traffic
3. A full test suite — generic, singular, and custom generic tests
4. A clean `dbt build` with zero failures

Checklist

- [] `macros/classify_revenue.sql` — macro with `if/elif/else` + default arguments
- [] `models/marts/mart_revenue_tiers.sql` — SQL model that uses the macro twice
- [] `models/marts/mart_channel_performance.py` — Pandas Python model
- [] `models/marts/_new_models.yml` — tests for both new models
- [] `tests/assert_channel_conversion_rate_is_valid.sql` — singular test
- [] `dbt build -s mart_revenue_tiers mart_channel_performance` passes with zero failures

Exercise 1 — Macro (20 min)

Create `macros/classify_revenue.sql`.

The macro takes an `amount_col` argument and returns a `case` expression that assigns `'low'`, `'medium'`, or `'high'` based on thresholds.

- Default thresholds: `low = 100`, `high = 500`
- Callers must be able to override the thresholds at the call site

Exercise 1 — Model (cont.)

Create `models/marts/mart_revenue_tiers.sql` using `int_orders_enriched` as the source.

Include only completed orders. Pass through `order_id`, `customer_id`, `order_date`, `status`, `total_amount`, and add:

Column	Description
<code>revenue_tier</code>	Classified with default thresholds
<code>revenue_tier_strict</code>	Classified with <code>low=50, high=200</code>

Exercise 2 — Python Model (25 min)

Create `models/marts/mart_channel_performance.py` using `stg_website_sessions`.

Available columns: `session_id`, `source`, `device_type`, `page_views`, `session_duration_seconds`, `is_bounce`, `converted`.

Group by `source` and `device_type`. The model must produce:

Column	Description
<code>source</code>	Traffic source
<code>device_type</code>	
<code>total_sessions</code>	Count of sessions
<code>conversions</code>	Count of converted sessions
<code>avg_page_views</code>	

Exercise 3 — Tests (15 min)

Generic tests — add `models/marts/_new_models.yml` with tests for both models:

- `mart_revenue_tiers`: `order_id` must be unique and not null; `revenue_tier` and `revenue_tier_strict` must be not null and only contain `'low'`, `'medium'`, `'high'`; `total_amount` must be positive
- `mart_channel_performance`: `source` and `device_type` must be not null and contain only known values; `conversion_rate` and `channel_quality` must be not null; `conversion_rate` must be positive

Singular test — create `tests/assert_channel_conversion_rate_is_valid.sql`:

- Business rule: `conversion_rate` must be between 0 and 100
- The query must return the rows that violate the rule (if any)

Verify

```
dbt build -s mart_revenue_tiers mart_channel_performance
```

All models and tests must pass.

To inspect the SQL your macro generated before running:

```
dbt compile -s mart_revenue_tiers
```


Bonus — Extend the Macro

Rewrite `classify_revenue` so that thresholds and labels are passed as **lists** instead of individual arguments.

The caller should be able to pass any number of thresholds and matching labels without changing the macro itself. Use a Jinja `for` loop to generate the `case` branches dynamically.

What have we practiced in this session

- **Macros:** `if/elif/else`, default arguments, overriding at call site
- **Python models:** `dbt.ref().df()`, Pandas groupby/agg, `.apply()` with a classification function
- **Tests:** generic, accepted values, custom generic (`is_positive`), singular business-rule test